

PROGRAMA DE RESIDÊNCIA EM TIC 16 — BRISA & UEMA

SIMPA

Sistema Integrado de Monitoramento de Projetos
Acadêmicos

Documentação Técnica do Projeto

Universidade Estadual do Maranhão (UEMA)

Pró-Reitoria de Extensão e Assuntos Estudantis (PROEXAE)

Equipe de Residentes TIC 16 — Turma 2 (2026)

Orientador: Prof. Me. Carlos Ronyhelton

Sumário

1. Visão Geral do Projeto
2. Equipe do Projeto
3. Objetivos e Funcionalidades Previstas
4. Tecnologias Utilizadas
5. Arquitetura do Sistema
6. Perfis de Usuário e Funcionalidades
7. Manual de Uso Rápido
8. Guia de Utilização — Página por Página
9. Modelo de Dados
10. Segurança e Auditoria
11. Guia de Instalação e Configuração
12. Estrutura de Diretórios do Repositório
13. Repositório e Contato

1. Visão Geral do Projeto

O **SIMPA** (Sistema Integrado de Monitoramento de Projetos Acadêmicos) é uma plataforma web desenvolvida para apoiar a gestão, o acompanhamento e a avaliação de projetos acadêmicos da Universidade Estadual do Maranhão (UEMA), no âmbito da Pró-Reitoria de Extensão e Assuntos Estudantis (PROEXAE).

Na prática, o sistema resolve um problema concreto de coordenação: projetos de extensão envolvem um orientador, um grupo de alunos e uma sequência de entregas com prazo (relatórios, produções, documentos comprobatórios). Antes de existir uma ferramenta como o SIMPA, esse acompanhamento dependia de e-mail, planilhas soltas e cobrança manual. O sistema centraliza essa informação em três painéis — administrador, professor e aluno — cada um com acesso apenas ao que lhe compete, e usa uma tabela de agenda (`agenda_items`) como espinha dorsal para gerar tanto o calendário do aluno quanto as notificações automáticas de prazo.

O projeto foi desenvolvido durante a etapa de imersão do **Programa de Residência em TIC 16**, parceria entre a **BRISA** e a **UEMA**, com apoio institucional da PROEXAE.

2. Equipe do Projeto

Equipe de Residentes do Programa de Residência em TIC 16 — BRISA e Softex Maranhão (2ª Turma, 2026).

Nome	Função
José Kauã	Fullstack Developer
Bruno Kauan	Fullstack Developer
Augusto Nicácio	Front-end Developer
Aian Arnaud	Back-end Developer
André Nunes	UI/UX Designer
Carlos Ronyhelton	Professor Orientador

3. Objetivos e Funcionalidades Previstas

O objetivo central do SIMPA é substituir o acompanhamento manual de projetos acadêmicos por um fluxo único e rastreável, cobrindo:

- **Gestão de projetos:** cadastro, edição e controle de status (pendente, ativo, concluído, inativo) de projetos e ações de extensão.
- **Monitoramento de prazos:** cada tarefa e evento vira um registro em `agenda_items`, com data, hora e prioridade, usado tanto no calendário quanto no motor de notificações.
- **Relatórios:** indicadores consolidados na tela e exportação em CSV ou HTML imprimível — não há geração de PDF binário, o "relatório" é uma página HTML com botão de impressão do navegador.
- **Gestão de participações:** vínculo de alunos e professores a projetos, com função (ex.: Orientador) e carga horária semanal.
- **Documentos e certificados:** upload, avaliação (aprovar / reprovar / solicitar correção) e emissão de certificados.
- **Responsividade:** as telas usam Bootstrap 5 com breakpoints padrão, sem layout separado para mobile — a mesma página se reorganiza conforme a largura da tela.

4. Tecnologias Utilizadas

Camada	Tecnologia
Front-end	HTML5, CSS3, JavaScript, Bootstrap 5, Bootstrap Icons, Chart.js (gráficos dos relatórios)
Back-end	PHP 8.2, sem framework — organização própria inspirada em MVC
Banco de dados	PostgreSQL, hospedado no Supabase, acessado via PDO (<code>pgsql:</code> , <code>sslmode=require</code>)
E-mail transacional	API HTTP da Resend, usada apenas no fluxo de redefinição de senha (envio do código de verificação)
Servidor web	Apache, com <code>mod_rewrite</code> habilitado
Containerização	Docker (<code>php:8.2-apache</code>)
Ambiente local alternativo	XAMPP (Apache + PHP)
Controle de versão	Git & GitHub

Nota de implementação: o `composer.json` do projeto declara a dependência `phpmailer/phpmailer`, mas nenhum arquivo do sistema a importa ou instancia. O envio de e-mail realmente usado está em `lib/Mailer.php`, que monta um payload JSON e faz uma chamada cURL direta para `api.resend.com/emails`, autenticada por `RESEND_API_KEY`. Essa variável, porém, não consta no `.env.example` do repositório (que só documenta variáveis `SMTP_*`) — vale corrigir isso antes da entrega final, junto com uma decisão sobre remover ou não a dependência do PHPMailer.

5. Arquitetura do Sistema

O SIMPA segue uma organização inspirada no padrão **MVC**, mas agrupada primeiro por **perfil de usuário** (administrador, professor, aluno) e só depois por camada. Isso significa que, em vez de um único diretório de controllers para o sistema inteiro, existem três — um por perfil — o que facilita aplicar regras de acesso distintas em cada um.

Camada	Diretório	Responsabilidade
Model	<code>/model</code>	Classes de acesso a dados (PDO) com SQL escrito à mão (sem ORM) — uma classe por entidade (Usuário, Projeto, Participação, Produção, Relatório, Acesso, Notificação, Aluno).
Controller	<code>/controllers/controller-adm</code> , <code>controller-professor</code> , <code>controller-aluno</code>	Endpoints PHP que recebem POST/GET, validam entrada, checam propriedade do recurso (ex.: se o professor participa do projeto) e devolvem JSON.
View	<code>/views/view-adm</code> , <code>view-aluno</code> , <code>view-professor</code>	Fragmentos PHP renderizados dentro das páginas de cada perfil — inclui, por exemplo, o cálculo do calendário do aluno feito diretamente na view, sem depender de JavaScript para montar a grade de dias.
Páginas / API interna	<code>/pages-adm</code> , <code>/pages-aluno</code> , <code>/pages-professor</code>	Páginas completas e endpoints AJAX (ex.: <code>api-projetos.php</code> , <code>toggle-concluido.php</code>) protegidos por sessão.
Infraestrutura	<code>/lib</code>	<code>Guard.php</code> (controle de acesso por sessão/perfil), <code>Logger.php</code> (auditoria em <code>logs_auditoria</code>), <code>Mailer.php</code> (envio via Resend).
Conexão	<code>/conexao/conexao.php</code>	Abre a conexão PDO lendo <code>DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASS</code> de variáveis de ambiente; em caso de falha, responde HTTP 500 sem expor detalhes da exceção ao cliente (só grava no <code>error_log</code>).

Fluxo de autenticação

O login é processado em `processa_login.php`: valida e-mail e senha contra a tabela `usuarios` (senha comparada via `password_verify`, já que é armazenada com `password_hash`), grava a sessão e registra o resultado — sucesso ou falha — na tabela `acessos`. Um detalhe específico do código: como o Supabase usa um tipo `ENUM status_acesso` para essa coluna, o script primeiro consulta `pg_enum/pg_type` para descobrir os valores exatos do enum (cacheados na sessão) antes de gravar, em vez de assumir que os literais `'sucesso' / 'falha'` existem. Depois do login, cada página protegida chama `Guard::autenticar()`, `Guard::apenasAdmin()` ou `Guard::apenasProfessor()`; se a sessão não existir, o usuário é redirecionado ao login — ou recebe um JSON de erro, se a requisição for AJAX (identificada pelo cabeçalho `X-Requested-With` ou `Accept: application/json`).

6. Perfis de Usuário e Funcionalidades

6.1 Administrador

Página inicial: `pages-adm/pagina-inicial.php`.

- Usuários: cadastro exige nome, e-mail e senha; matrícula e curso são opcionais; perfil padrão é `aluno` se não informado; cada usuário registra quem o cadastrou (`cadastrado_por`).
- Projetos: cadastro exige título e data de início; status transita entre `pendente`, `ativo`, `concluido` e `inativo`, e cada mudança gera uma entrada de auditoria com a ação correspondente (ATIVAR, DESATIVAR, APROVAR ao concluir, ou EDITAR nos demais casos).
- Participações: vincula usuários a projetos definindo função e carga horária semanal.
- Produções: aprova ou rejeita produções acadêmicas enviadas.
- Relatórios: indicadores consolidados na tela, com exportação em CSV (delimitado por `;`, com BOM UTF-8) ou relatório HTML pronto para impressão — ver Seção 8 para o funcionamento exato de cada tela, ou a Seção 7 para o passo a passo prático.
- Logs e visitas: consulta a tabela `logs_auditoria` com filtro por módulo, ação, usuário, texto livre e intervalo de datas.

6.2 Professor / Orientador

Página inicial: `pages-professor/pagina-inicial.php`.

- Só enxerga e só consegue agir sobre projetos onde participa — toda ação (criar tarefa, avaliar documento, emitir certificado) verifica a tabela `participacao` antes de executar, mesmo que o ID do recurso seja adivinhado na requisição.
- Cria tarefas para os alunos com título, data, hora e prioridade (alta / média / baixa).
- Avalia documentos enviados pelos alunos com três desfechos possíveis: aprovar, reprovar sem direito a correção, ou marcar como "refazer" (abre uma nova janela de reenvio para o aluno).
- Emite certificados em PDF/imagem para alunos participantes de projetos concluídos.

6.3 Aluno

Página inicial: `pages-aluno/pagina-inicial.php`.

É o perfil com o maior investimento de UX do sistema. Assim que o aluno autentica, a página inicial já carrega, lado a lado: a lista de tarefas e eventos (paginada de 2 em 2, com botões de navegação), um calendário do mês corrente e o total de carga horária acumulada. O calendário é montado inteiramente em PHP, no lado do servidor — a grade de dias, a marcação de "hoje" e as bolinhas coloridas por dia são calculadas antes de a página ser enviada ao navegador, e não dependem de nenhuma biblioteca de calendário em JavaScript. Cada dia recebe até quatro indicadores de cor conforme os itens daquela data: verde para tarefa/evento concluído, vermelho para atrasado, amarelo para vencimento em até 7 dias e azul para item futuro de baixa prioridade. Clicar em um dia abre um modal com a lista detalhada daquela data. Como o layout usa Bootstrap 5 sem breakpoints customizados, essa mesma tela se reorganiza em coluna única no celular sem precisar de uma versão separada.

- Cronograma: visão completa da agenda em lista, com o mesmo motor de cores do calendário.

- Tarefas: marcar como concluída, enviar arquivo de entrega, reenviar após correção solicitada — todas essas ações são bloqueadas pelo servidor se o prazo (data + hora) já tiver passado, e o "desfazer conclusão" também é bloqueado se o professor já tiver avaliado o documento.
- Projetos e participações: acompanha os projetos em que está inscrito, com orientador e demais participantes listados.
- Documentos, certificados, notificações e perfil: consulta e mantém seus próprios dados.

7. Manual de Uso Rápido

Este é o guia prático para quem vai operar o sistema no dia a dia — página por página, perfil por perfil, na mesma ordem em que cada item aparece no menu lateral. Cada entrada traz só o essencial: para que serve a tela e como usá-la sem errar. Para entender a lógica por trás de cada uma (regras, validações, o que cada botão dispara no servidor), veja a Seção 8.

7.1 Entrar no sistema

1. Acesse a página do SIMPA e clique em "**Acessar Sistema**".
2. Informe o e-mail e a senha cadastrados e clique em **Entrar**.
3. Você é levado direto ao seu painel — Administrador, Professor ou Aluno, conforme seu perfil.

Esqueceu a senha? Clique em "Esqueci minha senha" na tela de login, informe o e-mail cadastrado, digite o código de 6 dígitos recebido por e-mail (válido por 15 minutos) e defina uma nova senha com pelo menos 6 caracteres. Depois de 5 tentativas de código incorreto, será preciso solicitar um novo.

7.2 Administrador

Ordem do menu lateral: Página Inicial, Usuários, Participações, Projetos, Documentos, Visitas, Relatórios, Logs.

Página Inicial

Painel com contagens gerais — usuários, projetos ativos, pendências e acessos recentes. É só leitura: o ponto de partida para decidir para qual tela ir.

Usuários — cadastrar, editar, redefinir senha

1. Menu **Usuários** → botão "**Novo Usuário**".
2. Preencha nome, e-mail e senha (obrigatórios) e escolha o perfil: Aluno, Professor Orientador ou Administrador. Matrícula e curso são opcionais.
3. Salvar.

Para localizar alguém, use a busca (nome/e-mail/matricula) ou os filtros de status e perfil. Editar = ícone de lápis; trocar senha = ícone de chave (mínimo 6 caracteres); ativar/desativar = ícone de status, com confirmação antes de aplicar.

Participações — vincular aluno ou professor a um projeto

1. Menu **Participações** → aba "Visão por Projeto" → ícone de pessoa no card do projeto (ou "Vincular Usuário" no topo da página).
2. Escolha o tipo (Professor ou Aluno), selecione a pessoa e informe a função (ex.: "Orientador", "Bolsista") e a data de entrada. Carga horária semanal é opcional.
3. Salvar.

A aba "Todos os Vínculos" mostra a lista completa do sistema, com os mesmos filtros de busca/status. "Ver Equipe", em cada card de projeto, mostra os membros e um atalho para o relatório daquele projeto específico.

Projetos — cadastrar e mudar status

1. Menu **Projetos** → "**Novo Projeto**".
2. Preencha título e data de início (obrigatórios). Tipo, área, data de fim e descrição são opcionais.

3. Salvar. O projeto nasce como **Pendente** — use os botões de status na tabela para aprová-lo (vira Ativo) e, depois, concluí-lo.

Buscar por título/área/orientador ou filtrar por status/tipo no topo da tabela.

Documentos — revisar ou cadastrar manualmente

1. Menu **Documentos** — os quatro cartões no topo mostram total, aprovados, aguardando revisão e rejeitados.
2. Use a busca e os filtros de status/tipo para localizar um documento específico.
3. Para cadastrar um documento institucional sem esperar o envio de um aluno, use "Novo Documento": título, tipo e arquivo (até 10 MB; PDF, Word, Excel, PowerPoint, ZIP/RAR ou imagem).

Visitas (Monitoramento de Acessos)

Tela só de leitura com estatísticas de login: total de acessos, sucessos, falhas e usuários únicos. Use os botões de período (7, 30 ou 90 dias) para trocar a janela de análise — útil para notar picos de falha de login fora do padrão.

Relatórios — visualizar e exportar

- Menu **Relatórios** para ver os gráficos e tabelas na própria tela (sem cadastro).
- Menu **Participações** → "Exportar" para baixar em CSV (abre direto no Excel) ou gerar um relatório em HTML — que pode virar PDF pela opção de impressão do navegador.

Logs — consultar o histórico de ações

1. Menu **Logs** → use os filtros de módulo, ação, texto ou período, se precisar.
2. Botão **"Exportar CSV"** no topo baixa os resultados já filtrados.

É a única tela do sistema com paginação configurável (50 a 500 registros por página) em vez de carregar tudo de uma vez.

7.3 Professor

Ordem do menu lateral: Página Inicial, Meus Projetos, Meus Alunos, Tarefas, Cronograma, Documentos, Relatórios, Certificados.

Página Inicial

Cartões de projetos ativos, alunos orientados, documentos pendentes e tarefas vencendo, seguidos de "Hoje & Amanhã", "Atenção Necessária" (alunos com tarefas atrasadas) e "Documentos Pendentes" — este último permite aprovar ou reprovar um documento com um clique, direto na Página Inicial, sem abrir a tela de Documentos.

Meus Projetos

Lista os projetos em que você participa. Ponto de partida para abrir o cadastro de tarefas ou a avaliação de documentos de um projeto específico.

Meus Alunos

Lista todos os alunos vinculados aos seus projetos — nome, e-mail, matrícula, projeto, função e carga horária — com cartões de total, ativos e carga horária somada. Para adicionar um aluno a um dos seus projetos, use o botão correspondente; o menu de projetos mostra só aqueles em que você participa.

Tarefas — criar uma tarefa para um aluno

1. Menu **Tarefas** (ou Cronograma) → "Nova Tarefa".
2. Escolha o projeto e o aluno, defina título, data, hora (opcional) e prioridade (alta, média ou baixa).
3. Salvar — a tarefa já aparece automaticamente no calendário do aluno.

Só é possível criar ou editar tarefas em projetos onde você participa; o servidor confirma isso mesmo que o ID do projeto seja forçado na requisição.

Cronograma

Mostra a agenda de todos os seus projetos organizada por semana, complementando a visão de Tarefas — útil para enxergar rapidamente o que vence nos próximos dias.

Documentos — avaliar um envio

1. Menu **Documentos**, ou direto na Página Inicial, no bloco "Documentos Pendentes".
2. Clique em ✓ para aprovar, ✗ para reprovar, ou escolha "Solicitar correção" quando o aluno ainda pode reenviar.

Depois de aprovar ou reprovar um documento, o aluno não consegue mais alterar aquela entrega — confira antes de confirmar.

Relatórios

Mesmos indicadores e gráficos da tela de Relatórios do administrador, mas restritos aos seus próprios projetos.

Certificados — emitir

1. Menu **Certificados** → "Novo Certificado".
2. Escolha o projeto e o aluno (ele precisa já estar vinculado ao projeto), informe o título e anexe o arquivo.
3. Salvar — o certificado passa a aparecer para o aluno na área dele.

Só é possível emitir certificado em projetos onde você tem função de professor, coordenador ou orientador.

7.4 Aluno

Ordem do menu lateral: Página Inicial, Gerenciar Projetos, Registros, Minhas Tarefas, Cronograma, Documentos, Certificados.

Página Inicial — ver o que fazer na semana

Basta entrar: a Página Inicial já mostra o calendário do mês e a lista de tarefas e eventos. As bolinhas coloridas indicam o status de cada dia — **verde** concluído, **vermelho** atrasado, **amarelo** vence em até 7 dias, **azul** prazo mais distante. Clique em qualquer dia do calendário para ver os detalhes.

Gerenciar Projetos

Lista os projetos em que você participa, com tipo, função, orientador, carga horária e status. Clique em uma linha para ver a descrição completa, datas de início/fim e a lista de participantes (o orientador sempre aparece primeiro).

Registros

Histórico completo de tudo que já foi registrado na sua agenda — tarefas e eventos, passados e futuros —, com busca por título e estatísticas de total, concluídos, pendentes e itens do mês. Use esta tela para conferir algo antigo; para o que vence agora, prefira Cronograma ou Minhas Tarefas.

Minhas Tarefas — marcar como concluída ou enviar arquivo

1. Menu **Minhas Tarefas**.
2. Clique no ícone de check para concluir, ou no clipe/lápis para anexar um arquivo de entrega.

Só é possível desfazer uma conclusão antes do prazo vencer e antes de o professor avaliar o envio.

Minhas Tarefas — reenviar após correção solicitada

1. Itens marcados **"Corrigir"** mostram um botão de reenvio.
2. Anexe o novo arquivo e confirme.

O reenvio só funciona antes do prazo da tarefa vencer — depois disso, o botão aparece bloqueado.

Cronograma

A mesma agenda de Minhas Tarefas, organizada por semana (domingo a sábado), com os mesmos indicadores de status. Boa para planejar os próximos dias sem abrir o calendário mensal inteiro.

Documentos

Lista os arquivos que você mesmo enviou, com status (Aguardando Aprovação, Aprovado, Reprovado, Corrigir) e ícone conforme o tipo de arquivo. Só aparecem aqui documentos realmente salvos no servidor.

Certificados

Lista os certificados emitidos para você pelos professores, com opção de download. Os cartões no topo mostram o total de certificados e o número de projetos distintos com certificado emitido.

Notificações e Perfil (comuns aos três perfis, acessados pelo topo da página)

Notificações (ícone de sino): lista os avisos de prazo mais recentes primeiro — é recalculada a cada acesso, então não existe "marcar como lida". **Perfil** (nome/foto no canto superior): edite seus dados cadastrais e salve.

8. Guia de Utilização — Página por Página

As Seções 6 e 7 tratam do que cada perfil pode fazer e de como usar cada tela no dia a dia. Esta seção aprofunda o mesmo assunto **tela por tela** — os cartões de estatística, os filtros disponíveis e a lógica exata por trás de cada botão. As informações vêm diretamente das views e dos endpoints AJAX que elas chamam, não de uma descrição genérica de tela.

8.1 Acesso

O acesso começa em `login-page.php`. As credenciais (e-mail e senha) são enviadas a `processa_login.php`, que devolve um JSON de sucesso ou erro; no sucesso, o front-end redireciona para `adm-page.php`, `professor-page.php` ou `aluno-page.php` conforme o campo `perfil` da sessão. Não há "lembrar-me" nem token persistente — a sessão PHP nativa é o único mecanismo.

8.2 Administrador

Página Inicial

Painel com contagens gerais de usuários, projetos ativos, pendências e acessos recentes, servindo como ponto de partida para as demais telas.

Usuários

Quatro cartões no topo mostram total de usuários, ativos, inativos e administradores. Abaixo, uma barra de filtros permite buscar por nome/e-mail/matricula em tempo real (filtragem no navegador, sem nova requisição) e restringir por status (ativo/inativo) e por perfil (admin/professor_orientador/aluno). A tabela lista ID, nome (com avatar gerado automaticamente via ui-avatars.com), e-mail, matrícula, perfil e status; cada linha tem três ações — editar (abre um modal com os mesmos campos do cadastro, exceto senha), redefinir senha (modal separado, exige mínimo de 6 caracteres) e ativar/desativar (com confirmação via `confirm()` do navegador antes de chamar `api-usuarios.php?acao=alterarStatus`). O cadastro de um novo usuário pede nome, e-mail, senha, matrícula (opcional), perfil, curso (opcional) e status.

Projetos

Mesma lógica de cartões e filtros da tela de usuários, mas para projetos: total, ativos, pendentes e concluídos, com busca por título/área/orientador e filtro por status e por tipo. A tabela mostra também o orientador do projeto (calculado por uma subquery que busca o participante com função contendo "orientador") e o número de participantes. As ações mudam conforme o status atual: um projeto `pendente` ganha um botão "Aprovar" (muda para ativo); um projeto `ativo` ganha um botão "Concluir"; qualquer projeto que não esteja inativo pode ser desativado. Editar abre o mesmo modal do cadastro, pré-preenchido.

Participações

É a tela mais elaborada do painel administrativo, organizada em duas abas. A aba **"Visão por Projeto"** mostra um cartão por projeto (com contagem de membros ativos/total) e um botão "Ver Equipe" que abre um modal com duas sub-abas — Membros (tabela completa da equipe, carregada via `api-`

`participacoes.php?acao=listarPorProjeto`) e Produções e Atividades (um atalho para abrir o relatório HTML daquele projeto em nova aba). A aba **"Todos os Vínculos"** mostra uma tabela plana de todas as participações do sistema, com os mesmos filtros de busca/status/perfil das outras telas. Vincular um novo usuário exige escolher o projeto, o tipo de membro (professor ou aluno — a escolha troca o combo de seleção exibido), a função (texto livre, com sugestão automática de "Professor Orientador" ou "Bolsista") e a data de entrada; carga horária e data de saída são opcionais. Um menu "Exportar" no topo da página oferece relatório geral (CSV ou HTML) e relatório por projeto específico, escolhido em um modal à parte.

Relatórios

Página só de leitura, sem cadastro. Quatro cartões de resumo (projetos, usuários, participações, produções) seguidos de seis blocos com gráficos Chart.js: projetos por status (rosca), projetos por tipo (barras), novos projetos por mês (linha, últimos 12 meses), acessos por mês — sucesso vs. falha (linha, últimos 6 meses), usuários por perfil (rosca) e produções por tipo (linha). Fecha com duas tabelas: top 10 projetos por número de participantes e participações agrupadas por função, ambas com barra de progresso proporcional ao total. O botão "Imprimir" no topo apenas aciona `window.print()` — não existe exportação separada desta tela.

Logs de Auditoria

Seis cartões de estatística (total, logins OK, falhas de login, modificações, remoções, últimas 24h) vêm de `Logger::estatisticas()`. Os filtros cobrem busca textual (na descrição ou no nome do responsável), módulo, ação, data inicial e data final, com paginação configurável (50/100/200/500 registros por página). Cada linha da tabela mostra data/hora, um badge colorido por módulo, um badge colorido por ação, a descrição da ação com o contexto extra exibido como pequenas etiquetas, o responsável (com avatar) e o IP de origem. Um botão "Exportar CSV" replica os filtros atuais na exportação. Esta é a única tela do sistema que busca dados via `fetch` assíncrono com paginação numerada em vez de carregar tudo de uma vez — necessário porque o histórico de auditoria cresce indefinidamente.

8.3 Professor

Página Inicial

Painel bem mais denso que o do administrador. Quatro cartões no topo (projetos ativos, alunos orientados, documentos pendentes, tarefas vencendo) são seguidos de três blocos lado a lado: **"Hoje & Amanhã"** (lista de eventos e tarefas com prazo nesses dois dias, com uma barra colorida por tipo — reunião, prazo, entrega, evento, tarefa), **"Projetos por Tipo"** (gráfico de rosca da distribuição dos projetos ativos do professor) e **"Atenção Necessária"** (lista de alunos com tarefas atrasadas, marcados como "Urgente" quando há 3 ou mais atrasos ou pelo menos uma tarefa de prioridade alta). Abaixo, "Atividade Recente" mostra os últimos documentos enviados por alunos (com tempo relativo tipo "2 h atrás") e "Documentos Pendentes" permite aprovar ou reprovar um documento **direto da página inicial**, sem abrir a tela de Documentos — um clique chama `atualizar-status-doc.php` e remove o item da lista com uma transição de opacidade.

Meus Projetos / Alunos

Lista os projetos onde o professor participa e, por projeto, os alunos vinculados — usada como ponto de partida para abrir o cadastro de tarefas ou a avaliação de documentos de um aluno específico.

Cronograma e Tarefas

Formulário de criação/edição de tarefa (`salvar-tarefa.php`) com título, projeto, aluno, data, hora e prioridade (alta/média/baixa). Antes de gravar, o servidor confirma que o professor participa do projeto selecionado e, em caso de edição, que a tarefa pertence a um projeto onde ele participa — duas checagens independentes, então trocar o ID do projeto na requisição não basta para escrever em um projeto alheio.

Documentos

Lista os documentos enviados pelos alunos nos projetos do professor, com busca por título (usando `unaccent()` do Postgres, ou seja, buscar "relatorio" encontra "relatório") e filtro por status. Cada linha mostra um ícone de arquivo conforme a extensão (PDF em vermelho, planilha em verde, Word em azul) e um badge de status usando o mesmo vocabulário do aluno — pendente, aprovado (`concluido`), reprovado (`cancelado`) ou refazer. A avaliação em si tem dois caminhos no código: um atalho de dois botões (aprovar/reprovar, via `avaliar-doc-tarefa.php`) e uma via mais explícita que também permite marcar "refazer" (`atualizar-status-doc.php`).

Certificados

Lista os certificados já emitidos nos projetos do professor, cruzando a tabela `producoes` (filtrando pelos caminhos que começam com `uploads/certificados/aluno/`) com a matrícula do aluno para identificar o dono do arquivo. Emitir um novo certificado exige selecionar projeto, aluno e título, e só é aceito se o professor logado tiver função de professor/coordenador/orientador naquele projeto especificamente.

Relatórios

Indicadores e listagens restritos aos projetos do próprio professor — mesma lógica de exportação (CSV/HTML) descrita na Seção 8.2, mas sempre filtrada por participação.

8.4 Aluno

Página Inicial

Já detalhada na Seção 6.3 — calendário mensal renderizado em PHP, lista de tarefas/eventos paginada e carga horária acumulada.

Minhas Tarefas

A tela com a lógica de estado mais complexa do sistema. Quatro cartões mostram pendentes, a corrigir, não concluídas e concluídas. A tabela principal calcula o status de cada linha combinando três fontes: se `concluido` é verdadeiro, se o prazo (data, e hora quando presente) já passou, e se existe um documento em `producoes` com o mesmo título marcado como `refazer` — este último caso sempre tem prioridade e força o status "Corrigir", mesmo que a tarefa já estivesse marcada como concluída. O botão de ação de cada linha muda de acordo com essa combinação: tarefa em aberto dentro do prazo mostra

"marcar como concluída" e "anexar arquivo"; tarefa marcada para corrigir mostra "reenviar" e "enviar novo arquivo" (ou aparece travada com um ícone de cadeado, se o prazo já tiver passado); tarefa concluída e já avaliada pelo professor mostra só a opção de visualizar o arquivo enviado, sem permitir desfazer. Clicar em qualquer linha abre um modal de detalhe com a descrição completa e os arquivos anexados.

Cronograma

Mesmos quatro indicadores da tela de Tarefas (próximos, corrigir, não concluídas, concluídas), mas organizados por semana — o servidor calcula o domingo e o sábado da semana corrente e agrupa os itens por dia dentro desse intervalo, complementando a visão mensal do calendário da página inicial com uma visão semanal mais detalhada.

Gerenciar Projetos

Lista os projetos em que o aluno participa, com tipo, função, orientador, carga horária e status. Clicar em uma linha abre um painel de detalhe (via atributos `data-*` já embutidos na própria linha, sem nova requisição) com a descrição do projeto, datas de início/fim e a lista completa de participantes, obtida por uma subquery `json_agg` que já ordena o orientador primeiro.

Documentos, Certificados, Notificações e Perfil

Documentos: envio e acompanhamento de status dos arquivos enviados pelo aluno. Certificados: consulta e download dos certificados emitidos pelo professor. Notificações: lista gerada em tempo real a cada acesso à página (ver o cruzamento de seis situações descrito na Seção 6.3/Seção 10), não um feed persistente em banco. Perfil: edição de dados pessoais e foto do próprio usuário.

9. Modelo de Dados

O SIMPA usa PostgreSQL (hospedado no Supabase), acessado via PDO com SQL escrito manualmente — não há migrations nem ORM no repositório. As tabelas abaixo foram reconstruídas lendo as consultas de `/model` e dos controllers e, em seguida, conferidas contra o schema físico real (`pg_dump --schema-only`, reproduzido na Seção 11.4) — as divergências entre o que o código sugeria isoladamente e o que o banco realmente impõe estão anotadas nos campos correspondentes.

9.1 Entidades Principais

usuarios

Campo	Descrição
id_usuario	Chave primária
nome	Nome completo
email	E-mail (usado como login), com restrição <code>UNIQUE</code> no banco
senha	Hash bcrypt (<code>password_hash</code>)
matricula	Matrícula institucional, opcional
perfil	admin / professor / aluno
curso	Curso do usuário, opcional
status	ativo / inativo
cadastrado_por	ID do usuário que fez o cadastro — FK auto-referenciada para <code>usuarios.id_usuario</code>

projetos

Campo	Descrição
id_projeto	Chave primária
id_tipo	FK → <code>tipo_projetos</code>
titulo	Obrigatório no cadastro
area	Área de atuação
descricao	Descrição livre
data_inicio	Obrigatória no cadastro
data_fim	Opcional — projeto "em andamento" se vazia
status	pendente / ativo / concluido / inativo

tipo_projetos

Campo	Descrição
-------	-----------

id_tipo	Chave primária
nome	Nome do tipo de projeto
descricao	Descrição do tipo, opcional

participacao

Campo	Descrição
id_participacao	Chave primária
id_projeto	FK → projetos
id_usuario	FK → usuarios
funcao	Texto livre (ex.: "Orientador"), comparado com ILIKE nas checagens de permissão
carga_horaria	Horas semanais dedicadas
data_entrada / data_saida	Período de participação
status	ativo / inativo / concluido

producoes

Campo	Descrição
id_producao	Chave primária
id_projeto	FK → projetos
titulo	Casado por texto com o título da tarefa em agenda_items — não há FK direta entre as duas tabelas
tipo	Na prática, o nome original do arquivo enviado (ex.: "relatorio-final.pdf"), gravado assim pelos três pontos de upload do sistema — apesar do nome da coluna, não é uma categoria semântica. ProducaoModel::inserir() usa 'outro' como valor de reserva quando o campo vem vazio.
caminho	Caminho relativo do arquivo em /uploads
status	Reaproveita o mesmo enum status_geral de usuarios/projetos/participacao — não é um tipo próprio. Na prática, a aplicação só grava pendente, concluido, cancelado, refazer ou inativo nesta tabela.
data_registro	Data do envio

agenda_items — alimenta o calendário do aluno

Campo	Descrição
id	Chave primária uuid (gen_random_uuid()) — a única tabela do sistema que não usa inteiro sequencial
id_projeto	FK → projetos
id_usuario	FK → usuarios (o aluno dono do item)

titulo	Título da tarefa/evento
tipo	tarefa / evento — único campo de todo o sistema com <code>CHECK CONSTRAINT</code> explícito no banco (<code>tipo = ANY (ARRAY['tarefa','evento'])</code>)
data / hora	Prazo — a hora é opcional
prioridade	alta / media / baixa (padrão <code>media</code>), validado só em PHP, sem restrição no banco
descricao	Texto livre
concluido	Booleano marcado pelo próprio aluno
status_tarefa	Coluna própria (varchar, padrão <code>pendente</code>), atualizada por <code>Aluno::toggleConcluido()</code> ao marcar/desmarcar conclusão. As telas do professor (<code>buscar-tarefas.php</code> , <code>cronograma.php</code>) não leem essa coluna — recalculam um valor com o mesmo nome via subquery em <code>producoes</code> . Ou seja, há dois caminhos independentes para "status da tarefa" que coincidem no nome, mas nem sempre no valor.
created_at	Usado para detectar "tarefa nova" nas notificações (janela de 24h)

acessos

Campo	Descrição
id_acesso	Chave primária
id_usuario	FK → <code>usuarios</code> , nulo em falhas de login com e-mail inexistente
email	E-mail usado na tentativa
status	Tipo ENUM do Postgres (<code>status_acesso</code>) — os valores reais são lidos em runtime via <code>pg_enum</code> , não fixados no código
data	Timestamp do acesso

logs_auditoria

Campo	Descrição
id	Chave primária (BIGSERIAL)
criado_em	Timestamp do evento
modulo	USUARIOS / PROJETOS / PARTICIPACOES / DOCUMENTOS / PERFIL / LOGIN / SISTEMA
acao	CRIAR / EDITAR / EXCLUIR / ATIVAR / DESATIVAR / APROVAR / REJEITAR / ALTERAR_SENHA / VINCULAR / DESVINCULAR / LOGIN_OK / LOGIN_FALHA / EXPORTAR
descricao	Texto legível da ação
id_usuario	FK → <code>usuarios</code> , <code>ON DELETE SET NULL</code> (o log sobrevive à exclusão do usuário)
nome_usuario	Nome capturado no momento da ação (não depende de join posterior)
ip	Resolvido a partir de <code>HTTP_CLIENT_IP</code> / <code>X-Forwarded-For</code> / <code>X-Real-IP</code> / <code>REMOTE_ADDR</code> , nessa ordem

9.2 Relacionamentos

- `usuarios.cadastrado_por` → `usuarios.id_usuario` (auto-relacionamento — quem cadastrou cada usuário)
- `projetos.id_tipo` → `tipo_projetos.id_tipo`
- `participacao.id_projeto` → `projetos.id_projeto`
- `participacao.id_usuario` → `usuarios.id_usuario`
- `producoes.id_projeto` → `projetos.id_projeto` (vínculo com a tarefa de origem é feito por *título*, não por chave estrangeira)
- `agenda_items.id_projeto` → `projetos.id_projeto`
- `agenda_items.id_usuario` → `usuarios.id_usuario`
- `acessos.id_usuario` → `usuarios.id_usuario`
- `logs_auditoria.id_usuario` → `usuarios.id_usuario`

9.3 Vocabulário de status usado no código

Tabela	Coluna	Valores observados
usuarios	status	ativo, inativo
projetos	status	pendente, ativo, concluído, inativo
participacao	status	ativo, inativo, concluído
producoes	status	pendente, concluído (aprovado), cancelado (reprovado), refazer (correção solicitada), inativo (soft delete)
agenda_items	tipo	tarefa, evento
agenda_items	prioridade	alta, media, baixa

O schema real (Seção 11.4) mostra que `usuarios.status`, `projetos.status`, `participacao.status` e `producoes.status` usam todos o mesmo tipo enumerado do Postgres, `status_geral` — não quatro vocabulários independentes, como uma leitura isolada do código sugere. O banco garante que o valor gravado pertence ao enum; não garante qual subconjunto faz sentido em cada tabela (nada impede, a nível de banco, um projeto receber status `'refazer'`, que semanticamente só existe para produções) — essa restrição mais fina fica só na camada PHP, em listas fixas por controller. Já `agenda_items.tipo` tem um `CHECK CONSTRAINT` explícito no banco (Seção 11.4); `prioridade` continua sem nenhuma restrição além do código PHP.

10. Segurança e Auditoria

- **Sessão e rotas:** `lib/Guard.php` concentra o controle de acesso, com métodos por perfil (`autenticar` , `apenasAdmin` , `apenasProfesor`) e resposta em JSON quando a sessão expira em requisições AJAX.
- **Senhas:** hash bcrypt via `password_hash()` ; nunca comparadas em texto puro.
- **Verificação de propriedade:** em vez de confiar no perfil da sessão isoladamente, endpoints de professor confirmam a linha em `participacao` antes de liberar qualquer escrita — isto é, um professor não consegue avaliar documento ou emitir certificado de um projeto onde não está vinculado, mesmo manipulando o ID na requisição.
- **Uploads:** documentos, certificados e produções são salvos em subpastas por matrícula dentro de `/uploads` , com nome de arquivo gerado aleatoriamente (`bin2hex(random_bytes())`), extensão validada por lista branca (pdf, doc, docx, jpg, jpeg, png) e limite de 10 MB por arquivo.
- **Auditoria:** `lib/Logger.php` grava toda ação administrativa relevante em `logs_auditoria` ; se a gravação falhar, o erro não interrompe a operação — cai em um arquivo de fallback (`logs/fallback.log`) para não travar o sistema por causa do log.
- **Registro de acesso:** toda tentativa de login, com sucesso ou falha, é gravada em `acessos` , usada tanto para auditoria quanto para as estatísticas de acesso do painel administrativo.

10.1 Redefinição de senha

O fluxo de "esqueci minha senha" (`api-redefinir-senha.php`) é implementado em três passos dentro da mesma sessão PHP. No primeiro, o sistema gera um código numérico de 6 dígitos, válido por 15 minutos, e tenta enviá-lo por e-mail via Resend; se o envio falhar (API não configurada, fora do ar etc.), o código **não** é devolvido ao navegador — fica só no log do servidor, justamente para que uma falha de e-mail não vire uma forma de sequestrar a conta sem acesso à caixa de entrada. No segundo passo, o código é conferido com um limite de 5 tentativas por solicitação; ao exceder, a sessão de redefinição é invalidada e o usuário precisa solicitar um novo código. No terceiro passo, a nova senha exige no mínimo 6 caracteres e confirmação por igualdade, sendo então gravada com o mesmo hash bcrypt usado no cadastro normal.

10.2 Proteção na exportação de dados

Como descrito na Seção 8.2, a exportação em CSV neutraliza células que começam com `=` , `+` , `-` ou `@` , prefixando-as com um apóstrofo — uma mitigação direta contra *CSV/Formula Injection*, um vetor comum quando dados de usuário (como o título de um projeto) acabam em uma planilha aberta por um administrador.

11. Guia de Instalação e Configuração

11.1 Pré-requisitos

- PHP 8.2+ com extensão `pdo_pgsql`
- Composer
- Banco PostgreSQL (o projeto foi desenvolvido apontando para um projeto Supabase)
- Docker

11.2 Variáveis de Ambiente

Crie um arquivo `.env` na raiz do projeto. O `.env.example` versionado cobre a conexão com o banco e um bloco de variáveis SMTP que, na prática, não é lido por nenhum código atual — o envio de e-mail depende de `RESEND_API_KEY`, ausente do exemplo:

```
DB_HOST=<host do banco Postgres/Supabase>
DB_PORT=<porta, geralmente 5432>
DB_NAME=<nome do banco>
DB_USER=<usuário do banco>
DB_PASS=<senha do banco>

# Usado por lib/Mailer.php – não documentado no .env.example atual do repositório
RESEND_API_KEY=<chave da API da Resend>
SMTP_FROM=seuemail@dominio.com
SMTP_NAME=SIMPA - UEMA ProExae
```

11.3 Instalação via Docker

```
# 1. Clonar o repositório
git clone <url-do-repositorio>
cd SIMPA

# 2. Configurar o arquivo .env (ver seção 11.2)

# 3. Construir e subir o container
docker build -t simpa .
docker run -p 8080:80 --env-file .env simpa

# 4. Acessar
http://localhost:8080
```

O `Dockerfile` instala as extensões PHP necessárias (`pdo`, `pdo_pgsql`, `pgsql`), habilita o `mod_rewrite` do Apache e roda `composer install --no-dev` automaticamente durante o build.

11.4 Banco de Dados

O repositório só versiona o script de criação da tabela de auditoria (`logs/criar_tabela_logs.sql`); as demais tabelas viviam apenas no schema do Supabase usado em desenvolvimento, sem DDL equivalente no repositório. O bloco abaixo é o schema físico real, exportado com `pg_dump --schema-`

`only` — substitui a reconstrução por leitura de consultas que era a única fonte disponível na primeira versão desta documentação (Seção 9) e deve ser versionado junto ao projeto.

```
-- Schema físico real (public), exportado via pg_dump --schema-only.  
-- Ordem e constraints preservadas como no banco; tipos enumerados (status_geral,  
-- status_acesso, o enum de "perfil") aparecem como USER-DEFINED porque a definição  
-- de CREATE TYPE não faz parte de um dump --schema-only limitado a tabelas.
```

```
CREATE TABLE public.tipo_projetos (  
    id_tipo integer NOT NULL DEFAULT nextval('tipo_projetos_id_tipo_seq'::regclass),  
    nome character varying NOT NULL,  
    descricao text,  
    CONSTRAINT tipo_projetos_pkey PRIMARY KEY (id_tipo)  
);
```

```
CREATE TABLE public.usuarios (  
    id_usuario integer NOT NULL DEFAULT nextval('usuarios_id_usuario_seq'::regclass),  
    nome character varying NOT NULL,  
    email character varying NOT NULL UNIQUE,  
    senha character varying NOT NULL,  
    matricula character varying,  
    perfil USER-DEFINED NOT NULL,  
    curso character varying,  
    status USER-DEFINED DEFAULT 'ativo'::status_geral,  
    cadastrado_por integer,  
    CONSTRAINT usuarios_pkey PRIMARY KEY (id_usuario),  
    CONSTRAINT usuarios_cadastrado_por_fkey FOREIGN KEY (cadastrado_por) REFERENCES  
public.usuarios(id_usuario)  
);
```

```
CREATE TABLE public.acessos (  
    id_acesso integer NOT NULL DEFAULT nextval('acessos_id_acesso_seq'::regclass),  
    id_usuario integer,  
    data timestamp without time zone DEFAULT CURRENT_TIMESTAMP,  
    email character varying,  
    status USER-DEFINED NOT NULL,  
    CONSTRAINT acessos_pkey PRIMARY KEY (id_acesso),  
    CONSTRAINT acessos_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES  
public.usuarios(id_usuario)  
);
```

```
CREATE TABLE public.projetos (  
    id_projeto integer NOT NULL DEFAULT nextval('projetos_id_projeto_seq'::regclass),  
    id_tipo integer,  
    titulo character varying NOT NULL,  
    area character varying,  
    descricao text,  
    data_inicio date,  
    data_fim date,  
    status USER-DEFINED DEFAULT 'ativo'::status_geral,  
    CONSTRAINT projetos_pkey PRIMARY KEY (id_projeto),  
    CONSTRAINT projetos_id_tipo_fkey FOREIGN KEY (id_tipo) REFERENCES  
public.tipo_projetos(id_tipo)  
);
```

```
CREATE TABLE public.participacao (  

```



```

    id_participacao integer NOT NULL DEFAULT
nextval('participacao_id_participacao_seq'::regclass),
    id_projeto integer,
    id_usuario integer,
    funcao character varying NOT NULL,
    carga_horaria integer DEFAULT 0,
    data_entrada date,
    data_saida date,
    status USER-DEFINED DEFAULT 'ativo'::status_geral,
    CONSTRAINT participacao_pkey PRIMARY KEY (id_participacao),
    CONSTRAINT participacao_id_projeto_fkey FOREIGN KEY (id_projeto) REFERENCES
public.projetos(id_projeto),
    CONSTRAINT participacao_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES
public.usuarios(id_usuario)
);

CREATE TABLE public.producoes (
    id_producao integer NOT NULL DEFAULT nextval('producoes_id_producao_seq'::regclass),
    id_projeto integer,
    titulo character varying NOT NULL,
    tipo character varying,
    caminho character varying,
    data_registro timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    status USER-DEFINED DEFAULT 'ativo'::status_geral,
    CONSTRAINT producoes_pkey PRIMARY KEY (id_producao),
    CONSTRAINT producoes_id_projeto_fkey FOREIGN KEY (id_projeto) REFERENCES
public.projetos(id_projeto)
);

CREATE TABLE public.agenda_items (
    id uuid NOT NULL DEFAULT gen_random_uuid(),
    titulo text NOT NULL,
    descricao text,
    tipo text NOT NULL CHECK (tipo = ANY (ARRAY['tarefa'::text, 'evento'::text])),
    data date,
    hora time without time zone,
    concluido boolean DEFAULT false,
    created_at timestamp with time zone DEFAULT now(),
    id_usuario integer,
    prioridade character varying DEFAULT 'media'::character varying,
    status_tarefa character varying DEFAULT 'pendente'::character varying,
    id_projeto integer,
    CONSTRAINT agenda_items_pkey PRIMARY KEY (id),
    CONSTRAINT agenda_items_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES
public.usuarios(id_usuario),
    CONSTRAINT agenda_items_id_projeto_fkey FOREIGN KEY (id_projeto) REFERENCES
public.projetos(id_projeto)
);

CREATE TABLE public.logs_auditoria (
    id bigint NOT NULL DEFAULT nextval('logs_auditoria_id_seq'::regclass),
    criado_em timestamp with time zone NOT NULL DEFAULT now(),
    modulo character varying NOT NULL,
    acao character varying NOT NULL,
    descricao text NOT NULL,
    id_usuario integer,
    nome_usuario character varying,

```

```
ip character varying,  
contexto jsonb,  
CONSTRAINT logs_auditoria_pkey PRIMARY KEY (id),  
CONSTRAINT logs_auditoria_id_usuario_fkey FOREIGN KEY (id_usuario) REFERENCES  
public.usuarios(id_usuario)  
);
```

Três achados deste schema que corrigem ou refinam a leitura feita só pelo código (Seção 9): (1) `agenda_items.id` é `uuid`, não inteiro — a única chave primária não sequencial do sistema; (2) `usuarios.status`, `projetos.status`, `participacao.status` e `producoes.status` compartilham um único enum (`status_geral`), todos com `DEFAULT 'ativo'` no banco — a aplicação é quem sobrescreve esse padrão (um projeto novo nasce `'pendente'` porque `projetoController.php` define isso explicitamente na criação, não porque o banco assim o exige); (3) `producoes.tipo` não tem restrição alguma no banco e, na prática, recebe o nome original do arquivo enviado, não uma categoria. A lista exata de valores válidos de `status_geral`, `status_acesso` e do enum de `perfil` não está neste dump — precisaria de `\dT+ status_geral` (ou consulta a `pg_enum`) para documentar formalmente.

12. Estrutura de Diretórios do Repositório

```
SIMPA/
├── assets/                # CSS, JS e imagens
├── conexao/conexao.php    # Conexão PDO com PostgreSQL/Supabase
├── controllers/
│   ├── controller-adm/
│   ├── controller-professor/
│   └── controller-aluno/
├── lib/                  # Guard.php, Logger.php, Mailer.php
├── logs/                 # Logs de aplicação e script SQL de auditoria
├── model/                # Classes de acesso a dados
├── pages-adm/            pages-professor/    pages-aluno/
│                           # Páginas e APIs por perfil
├── views/
│   ├── view-adm/
│   ├── view-professor/
│   └── view-aluno/
├── uploads/              # Documentos, certificados e produções
├── index.php             # Portal público
├── login-page.php        # Autenticação
├── processa_login.php    # Processamento de login
├── adm-page.php / professor-page.php / aluno-page.php
├── sobre.php             # Página institucional "Sobre"
├── composer.json         # Dependência declarada: phpmailer/phpmailer (não utilizada)
├── Dockerfile
├── .env.example
└── readme.md
```

13. Repositório e Contato

O código-fonte completo, incluindo este material de documentação, o arquivo `README` com orientações de instalação e o modelo de dados físico, deve ser disponibilizado através de um repositório Git, com acesso liberado para o e-mail institucional da BRISA responsável pelo recebimento da documentação final do projeto.

Equipe SIMPA — Residência TIC 16 (BRISA & UEMA)

José Kauã · Bruno Kauan · Augusto Nicácio · Aian Arnaud · André Nunes

Orientador: Prof. Carlos Ronyhelton

Instituição parceira: PROEXAE — UEMA
